

SynectivSec: A Composition Theorem for Federated Agent Authority

Abhijith S*

September 2026

Abstract

Delegation chains, sequence policies, and cross-domain federation each have formal treatments in the security and distributed-systems literature, but no result states when their *composition* preserves the safety properties of its parts. We give three compatibility conditions under which it does, prove the preservation theorem by structural induction over the transition relation, and show that dropping any one condition admits a concrete violation. We model-check the construction in TLA+ on a bounded instance (11,185,890 states generated, 300,447 distinct, zero violations of the five stated invariants) and state exactly where the proof and the verification stop. The individual conditions are not novel—each restates prior work—but the composition operator, the joint formulation of the conditions, and the preservation theorem are. The artifact is released at <https://github.com/kakashi-kx/synectivsec>.

1 Introduction

Autonomous AI agents take actions: they call tools, move funds, send communications, and invoke other agents. Each action is the composition of several layers of authority and policy. A delegation chain establishes how authority attenuates from a root principal to the acting agent. A sequence policy constrains the order and multiplicity of actions over time. A federation graph determines which trust domains accept whose attestations and how revocation propagates across organizational boundaries.

Each of these layers has been studied separately. Delegation chains are formalized in the access-control literature as far back as Lampson, Abadi, Burrows, and Wobber [1] and in the SPKI/SDSI certificate theory [3]; more recently, in the delegation-chain calculi of the agent-security literature [11]. Sequence policies over action histories are grounded in Schneider’s work on enforceable security policies [5] and in modern agent frameworks such as the Agentic Principal Chain [12]. Cross-domain federation is discussed in federated governance testbeds [18] and in the meta-framework COA-MAS v2 [19].

What is missing is a *composition theorem*: a statement of the conditions under which the composition of all three layers preserves the safety invariants of each. This absence is not accidental. Compositionality of security properties is difficult in general: McLean [8] showed that safety is not preserved under composition without strong assumptions, and recent work in agent security has made the difficulty concrete. Spera [13] proves a non-compositionality result showing that two individually safe agents can jointly reach a forbidden state through an

*Independent Researcher.
ORCID: 0009-0009-9232-7820.
GitHub: @kakashi-kx.
Artifact: github.com/kakashi-kx/synectivsec.

emergent conjunctive dependency. AgentRFC [14] documents 20 cross-protocol violations across five composition patterns.

The difficulty is real, but the composition question is not hopeless. We give a positive result for a specific class of systems: those satisfying three compatibility conditions. Under these conditions, the composition preserves five safety invariants. We prove the theorem by structural induction over the transition relation and verify it in TLA+ on a bounded instance. We further show that each condition is non-redundant: dropping any one admits a counterexample.

Contributions.

1. **The composition operator** $\otimes$ for the joint treatment of delegation chains, sequence policies, and federated attestation (§3).
2. **Three compatibility conditions**—Policy Compatibility (PC), Revocation Synchronization (RS), and Cross-Boundary Attestation Transitivity (CBAT)—as a sufficient set for the preservation of five safety invariants (§4).
3. **A preservation theorem** (Theorem 7) proved by structural induction over the transition relation (§5).
4. **A mechanized verification** in TLA+ on a bounded instance, with the raw output and reproduction instructions (§6).
5. **An independence result**: each condition is non-redundant, with a runnable counterexample per condition (§7).

What is not claimed. We do not claim novelty for any of the three conditions in isolation. CBAT restates the *speaks-for* transitivity of Lampson et al. [1] with the bounded variant the same paper suggests. RS applies the validity-interval concept of SPKI RFC 2693 [3] to a domain graph with an explicit propagation bound. PC relates to the sequence-policy work of Bounded Agents [12] and Schneider [5]. The claimed contribution is the *composition* of the three conditions, not their individual content. A full lineage is given in Table 2.

Paper organization. Section 2 presents the adversary model that motivates the composition question. Section 3 defines the three layers and the composition operator. Section 4 states the three compatibility conditions and the preservation theorem, with a per-condition lineage table. Section 5 gives the proof. Section 6 reports the TLA+ verification. Section 7 reports the independence result and the attack harness. Section 8 discusses related work. Section 9 states limitations and threats to validity. Section 10 concludes. The appendices contain the full proof (Section A) and the TLA+ module (Section B).

2 Composition adversaries

The adversaries that motivate this work are not the classical single-layer attackers. They exist only because the three layers compose. We catalogue them by the composition-specific capability each requires, following the attack catalog in the artifact’s `docs/ALGEBRA.md` and `redteam/README.md`.

Definition 1 (Composition adversary). A *composition adversary* is an attacker whose power derives from the interaction of the delegation, sequence, and federation layers, and whose goal is to cause an action to occur that no single layer would authorize.

Table 1 lists the eight adversary classes we consider. Each is characterized by the composition-specific capability it requires and the invariant it seeks to violate.

ID	Capability	Goal	Invariant at risk
AC-1	Holds a valid delegation chain	Extend it beyond intent, relying on the sequence policy not seeing the extension	CI2, CI4
AC-2	Controls two chains	Interleave actions such that no single chain’s sequence policy fires	CI2
AC-3	Holds attestations in two domains	Laundry authority across non-transitive attestation paths	CI4
AC-4	Controls timing across domains	Act in the window before revocation propagates	CI3
AC-5	Can craft action parameters	Produce an action outside the sequence policy’s recognized alphabet	CI2
AC-6	Observes one domain’s attestations	Replay an attestation in a domain where it was not issued	CI4
AC-7	Controls multiple agents	Jointly reach a forbidden state, each individually authorized	CI1–CI5
AC-8	Exploits trust-boundary ambiguity	Cause two domains to each believe the other’s policy applies	CI5

Table 1: Composition adversary classes. Each exists only under composition.

The key observation is that AC-1 through AC-8 are not hypothetical. Each corresponds to an attack that has been demonstrated in the agent-security literature or is a natural extension of one. AC-7 (emergent conjunctive authority) is the Spera non-compositionality result [13]. AC-3 and AC-6 (attestation laundering and replay) are natural cross-domain analogues of attacks documented in the trust-management literature. AC-4 (revocation race) is the composition of the classic time-of-check/time-of-use problem with cross-domain propagation. AC-5 (alphabet escape) is specific to the composition of the sequence layer with delegation.

Against this adversary set, the composition question is: under what conditions does the composition of the three layers prevent all of AC-1 through AC-8?

3 Model

We define the three layers, the composition operator, and the safety invariants. The definitions here are the specification; the artifact’s `docs/ALGEBRA.md` mirrors them, and `tla/Composition.tla` implements them in TLA+.

3.1 Principals and capabilities

A *principal* is a named entity that can sign statements, be delegated authority, and perform actions. Write `Principal` for the finite set of principals in a system, and $p, q, r, \dots$ for elements of it.

A *capability* is an atomic permission. Write `Capability` for the finite set of capabilities, and $c, c', \dots$ for elements. Capabilities form a lattice $\langle \text{Capability}, \sqsubseteq \rangle$ in which $c \sqsubseteq c'$ means c is at least as strong as c' (i.e., c permits everything c' permits, and possibly more). Each principal p has a capability set $\kappa(p) \subseteq \text{Capability}$.

3.2 Delegation chains

A *delegation chain* is a finite sequence

$$D = \langle \kappa_0, \sigma_0, \kappa_1, \sigma_1, \dots, \kappa_n, \sigma_n \rangle$$

where each κ_i is a capability set, each σ_i is a signature by the holder of κ_{i-1} attesting to κ_i , and the attenuation condition holds:

$$\kappa_n \sqsubseteq \kappa_{n-1} \sqsubseteq \dots \sqsubseteq \kappa_0.$$

The root capability set κ_0 is granted by a designated root principal. Let $\text{Reach}(D)$ be the set of actions producible under D without violating attenuation.

Prior art. The delegation-chain model is standard. Lampson, Abadi, Burrows, and Wobber [1] introduce the *speaks-for* relation and the handoff axiom in §3.2 and §3.3 respectively. SPKI RFC 2693 [3] formalizes certificate-chain reduction in §6.3. SentinelAgent [11] provides a modern delegation chain calculus with TLA+ verification. The definition above restates the standard model; it does not introduce new primitives.

3.3 Sequence policies

A *sequence context* is a pair $S = \langle \Sigma, \varphi \rangle$ where Σ is a finite *action alphabet* and $\varphi : \Sigma^* \rightarrow \{\top, \perp\}$ is a decidable predicate on finite action histories. We write $h \models \varphi$ for $\varphi(h) = \top$.

We assume φ is *prefix-closed*: $h \models \varphi$ implies $\text{prefix}(h, i) \models \varphi$ for all $i \leq |h|$. Prefix-closure characterizes the safety policies in the sense of Alpern and Schneider [6]. Liveness policies are outside the scope of this paper.

Let $\text{Reach}(S)$ be the set of histories $h \in \Sigma^*$ with $h \models \varphi$.

Prior art. Runtime enforcement of safety policies is Schneider’s [5]. The specific formalism of sequence policies over agent actions is from Bounded Agents [12] (the Agentic Principal Chain). The definition above restates the standard model.

3.4 Trust domains and federation

A *trust domain* is a named region of policy and trust authority, written T or T_i . Let Domain be the set of all domains. A *federation graph* is a tuple

$$F = \langle \mathcal{T}, \mathcal{A}, \delta \rangle$$

where:

- $\mathcal{T} \subseteq \text{Domain}$ is a finite set of domains;
- $\mathcal{A} \subseteq \mathcal{T} \times \mathcal{T}$ is the *attestation relation*, with $\mathcal{A}(T_i, T_j)$ meaning “ T_i attests the claims of T_j ”;
- $\delta \in \mathbb{N}$ is the *revocation propagation bound*.

Let $\text{Reach}(F, T)$ be the set of domains reachable from T via $\mathcal{A}$, including T itself. We define Reach as the transitive closure of $\mathcal{A}$ together with the reflexive base case:

$$\text{Reach}(F, T) = \{T\} \cup \{T' : \mathcal{A}(T, T')\} \cup \{T'' : \exists T' . \mathcal{A}(T, T') \wedge \mathcal{A}(T', T'')\} \cup \dots$$

The revocation propagation bound δ is discussed in §4.

Prior art. Cross-domain trust is discussed in Wang’s federated governance testbed [18] and COA-MAS v2 [19]. Attestation transitivity is from Lampson et al. [1] §3.2 and SPKI RFC 2693 [3] §6.3. The definition above restates the standard model.

3.5 Composition states

A *composition state* is a tuple

$$s = \langle d, h, T, r, t \rangle$$

where:

- $d \in \text{Reach}(D)$ is the current position in a delegation chain;
- $h \in \Sigma^*$ is the current action history;
- $T \in \mathcal{T}$ is the current trust domain;
- $r : \text{Principal} \rightarrow \mathbb{N} \cup \{\infty\}$ maps each principal to its revocation time (or ∞ if never revoked);
- $t \in \mathbb{N}$ is the current logical time.

Write $\text{States}(D, S, F)$ for the set of all composition states.

3.6 The transition relation

The transition relation $\rightarrow$ on composition states is given by three rules.

Action transition. A principal p performs an action $a \in \Sigma$ in domain T :

$$\langle d, h, T, r, t \rangle \rightarrow \langle d, h \frown \langle \langle p, a, t \rangle \rangle, T, r, t + 1 \rangle$$

subject to five guards:

- G1 (Reachability): $T \in \text{Reach}(F, \text{RootDomain}(p))$
- G2 (Authorization): $a \in \kappa(p) \wedge t < r[p]$
- G3 (Sequence): $h \frown \langle a \rangle \models \varphi$
- G4 (Length): $|h| < \text{MaxHistory}$
- G5 (Boundary): $\text{DomainPolicy}(T, a) = \top$

Revocation transition. A principal p is revoked at time t :

$$\langle d, h, T, r, t \rangle \rightarrow \langle d, h, T, r[p \mapsto t], t + 1 \rangle$$

subject to $r[p] = \infty$ (not already revoked).

Domain transition. A principal p moves to a domain $T' \in \mathcal{T}$:

$$\langle d, h, T, r, t \rangle \rightarrow \langle d, h, T', r, t + 1 \rangle$$

subject to $T' \in \text{Reach}(F, \text{RootDomain}(p))$.

The TLA+ model in `tla/Composition.tla` implements these rules as `DoAction`, `Revoke`, and `CrossDomain` respectively.

3.7 The composition operator $\otimes$

Definition 2 (Composition). The composition $D \otimes S \otimes F$ is the labelled transition system $\langle \text{States}, \rightarrow, s_0 \rangle$ where:

- $\text{States} = \text{Reach}(D) \times \Sigma^* \times \mathcal{T}$ (with the revocation map and time implicit in the execution);
- $\rightarrow$ is the transition relation from §3;
- $s_0 = \langle d_0, \varepsilon, T_0, r_0, 0 \rangle$ is the initial state, with $r_0[p] = \infty$ for all p .

The *reachable set* $\text{Reach}(D \otimes S \otimes F)$ is the transitive closure of $\rightarrow$ from s_0 .

The operator $\otimes$ is binary and associative by construction: $(D \otimes S) \otimes F = D \otimes (S \otimes F)$. It is *not commutative* in general: $D \otimes S \neq S \otimes D$, because the sequence policy constrains actions, and actions are only produced under delegation. This order-sensitivity mirrors the Composition Safety principle of AgentRFC [14].

3.8 Safety invariants

We state five invariants on composition states. Write $h[i] = \langle p_i, a_i, t_i \rangle$ for the i -th entry of the history.

Definition 3 (CI1–CI5). For a state $s = \langle d, h, T, r, t \rangle$:

$$\begin{aligned} \text{CI1}(s) &= \forall i \in 1..|h|. \exists p \in \text{Principal}. h[i].\text{action} \in \kappa(p) \\ \text{CI2}(s) &= h \models \varphi \\ \text{CI3}(s) &= \forall i \in 1..|h|. h[i].\text{time} < r[h[i].\text{principal}] \\ \text{CI4}(s) &= T \in \text{Reach}(F, \text{RootDomain}(\text{currentPrincipal})) \\ \text{CI5}(s) &= \forall T \in \mathcal{T}, a \in \Sigma. \text{DomainPolicy}(T, a) = \top \vee \text{DomainPolicy}(T, a) = \perp \end{aligned}$$

Informally:

- **CI1 (Authority containment)**: every action in history is authorized by some principal’s capability set.
- **CI2 (Sequence soundness)**: the action history satisfies the sequence policy.
- **CI3 (Revocation freshness)**: no action follows its actor’s revocation.
- **CI4 (Attestation soundness)**: the current principal acts only in domains reachable from its root domain.
- **CI5 (Boundary determinism)**: the applicable domain policy is defined at every state.

4 Compatibility conditions and the preservation theorem

Three compatibility conditions, taken jointly, ensure the preservation of the five invariants. Each condition is stated below, followed by its lineage to prior work.

4.1 The conditions

Definition 4 (Policy Compatibility, PC). The composition $D \otimes S \otimes F$ satisfies *Policy Compatibility* if every action producible under any chain in D is classified by the sequence policy’s alphabet:

$$\text{PC}(D, S) = \forall d \in \text{Reach}(D). \text{Actions}(d) \subseteq \Sigma.$$

Cond.	Restates	Where
PC	Scope/budget tracking across action sequences; runtime enforcement of safety policies	Bounded Agents [12]; Schneider [5]
RS	Validity intervals on certificates; the question “how long are you willing to let the world believe and act on a statement you know to be false?”	SPKI RFC 2693 [3], §5.4
CBAT	Speaks-for transitivity: “it is also easy to show that $\vdash$ is monotonic in both arguments and that $\Rightarrow$ is transitive”	Lampson et al. [1], §3.2
CBAT	“the general axiom is too powerful... if the (bound-conclusion uses a qualified form of $\Rightarrow$ it may be more acceptable”; closest formal analog is RT’s linked roles	Lampson et al. [1], §3.3 (note on P10); Li, Mitchell, Winsborough [4]

Table 2: Prior-art lineage of the three compatibility conditions. Each condition restates prior work; the contribution is the joint treatment.

PC ensures that the sequence policy sees every action that can occur. Without it, an action outside Σ escapes the sequence constraint entirely.

Definition 5 (Revocation Synchronization, RS). The composition satisfies *Revocation Synchronization* with bound δ if revocation in one domain is effective in every domain reachable by an attestation path, within δ time units:

$$\text{RS}(F, \delta) = \forall T_i, T_j \in \mathcal{T}. \text{path}(\mathcal{A}, T_i, T_j) \Rightarrow \text{revoke}(T_i, t) \Rightarrow \text{effective}(T_j, t + \delta).$$

RS ensures that a revoked principal cannot continue to act in a domain that has not yet seen the revocation. Without it, the propagation window admits a race.

Definition 6 (Cross-Boundary Attestation Transitivity, CBAT). The composition satisfies *Cross-Boundary Attestation Transitivity* if attestation is transitive across domains unless explicitly bounded:

$$\text{CBAT}(F) = \forall T_i, T_j, T_k \in \mathcal{T}. \mathcal{A}(T_i, T_j) \wedge \mathcal{A}(T_j, T_k) \Rightarrow \mathcal{A}(T_i, T_k) \vee \text{explicitly_bounded}(T_i, T_k).$$

CBAT ensures that reachability is closed under composition of attestations. Without it, an agent can act in a domain reachable only through an intermediate domain whose attestation was not transitively closed—an attestation laundering attack.

4.2 Lineage of the conditions

None of the three conditions is novel in isolation. Table 2 states, for each condition, the prior work it restates and the aspect (if any) that extends or refines that prior work.

4.3 The preservation theorem

Theorem 7 (Composition Preservation). *If $\text{PC}(D, S) \wedge \text{RS}(F, \delta) \wedge \text{CBAT}(F)$ hold, then for every state $s \in \text{Reach}(D \otimes S \otimes F)$ and every $k \in \{1, \dots, 5\}$:*

$$\text{Clk}(s).$$

The contrapositive gives a concrete attack surface per condition:

Theorem 8 (Emergent Authority, contrapositive). *If $\neg\text{PC} \vee \neg\text{RS} \vee \neg\text{CBAT}$, there exists a trajectory τ in $\text{Reach}(D \otimes S \otimes F)$ that violates at least one of $\text{CI1}, \dots, \text{CI5}$.*

Theorem 8 is the red-teamer’s reading of the theorem: every failure of a compatibility condition yields a concrete attack. Section 7 demonstrates this for each condition by a runnable counterexample.

5 Proof of the preservation theorem

The proof is by structural induction over the transition relation. The base case establishes all five invariants at the initial state. The inductive step shows that each of the three transition rules preserves all five invariants. The full proof is given in Section A; here we give the induction outline and state where each compatibility condition is used.

5.1 Proof outline

Proof of Theorem 7, by structural induction. Base case. Let $s_0 = \langle d_0, \varepsilon, T_0, r_0, 0 \rangle$. Since $h = \varepsilon$, the quantifiers in CI1 , CI2 , and CI3 are vacuous. CI4 holds because $T_0 \in \text{Reach}(F, T_0) = \text{Reach}(F, \text{RootDomain}(p_0))$ by the reflexive base case of Reach . CI5 holds because DomainPolicy is total by definition.

Inductive step. Assume $\text{CI1}(s) \wedge \dots \wedge \text{CI5}(s)$ for some $s \in \text{Reach}(D \otimes S \otimes F)$. Let $s \rightarrow s'$. We consider the three transition rules in turn.

T-Action: $s' = \langle d, h \frown \langle \langle p, a, t \rangle \rangle, T, r, t + 1 \rangle$.

- CI1 follows from $\text{CI1}(s)$ for old history entries and from guard G2 for the new entry.
- CI2 follows directly from guard G3.
- CI3 follows from $\text{CI3}(s)$ for old entries and from guard G2 ($t < r[p]$) for the new entry. **RS is used here:** the value $r[p]$ must be consistent across domains, which is exactly what RS guarantees.
- CI4 follows from guard G1. **CBAT is used here:** the reachability relation Reach is defined as the transitive closure of $\mathcal{A}$, and CBAT is what makes the transitive closure well-defined across domains.
- CI5 follows from $\text{CI5}(s)$ and from guard G5. **PC is used here:** PC ensures the new action is in Σ , so the domain policy classification applies.

T-Revoke: $s' = \langle d, h, T, r[p \mapsto t], t + 1 \rangle$. The history, current principal, and current domain are unchanged. Only r changes, and only at p . The invariants CI1 , CI2 , CI4 , CI5 are immediate. For CI3 : for i with $h[i].\text{principal} \neq p$, the inequality is preserved; for i with $h[i].\text{principal} = p$, we have $h[i].\text{time} < t$ because no action in h is recorded at the current time (actions increment t), so the inequality holds.

T-Cross: $s' = \langle p, h, T', r, t + 1 \rangle$. The history is unchanged. CI1 , CI2 , CI3 are preserved. CI4 follows from the transition guard $T' \in \text{Reach}(F, \text{RootDomain}(p))$; **CBAT is used here** for the same reason as in T-Action. CI5 is unchanged. $\square$

5.2 Where each condition is used

Table 3 summarizes the use of each compatibility condition in the proof.

The use is disjoint: each condition is used in a distinct part of the preservation argument. This is what makes the conditions non-redundant, as Section 7 demonstrates by counterexample.

Cond.	Used in	Why
PC	CI2, CI5 preservation under T-Action	Ensures the new action is in Σ , so the sequence policy and domain policy classify it
RS	CI3 preservation under T-Action	Ensures revocation time $r[p]$ is consistent across domains, so the guard $t < r[p]$ is sound
CBAT	CI4 preservation under T-Action and T-Cross	Ensures $\text{Reach}(F, T)$ is the transitive closure of $\mathcal{A}$, so the guard $T \in \text{Reach}$ is well-defined

Table 3: Dependence of the proof on each compatibility condition.

Parameter	Value
Principals	6 (R_1, PA, PB, R_2, PC, PD)
Delegation chains	2 (chain D_1 rooted in T_1 ; chain D_2 rooted in T_2)
Trust domains	3 (T_1, T_2, T_3)
Capabilities	$\{\text{read}, \text{write}, \text{send}\}$
Sequence policy	No two consecutive send actions
Attestation relation	$T_1 \rightarrow T_2, T_2 \rightarrow T_3$
State constraint	$\text{time} \leq 4, \text{Len}(\text{history}) \leq 3$
Invariants checked	CI1, CI2, CI3, CI4, CI5

Table 4: The verified instance.

6 Mechanized verification

The theorem is verified in TLA+ for a bounded instance. The verification is not a proof of the general theorem—the proof in §5 carries that—but it catches specification bugs and confirms that the model is coherent on a concrete configuration.

6.1 The verified instance

The instance is deliberately minimal: it is small enough that TLC exhausts the reachable state space in seconds, and rich enough that each invariant is non-trivial. The sequence policy exercises two-chain interleaving. The attestation relation exercises transitivity from T_1 to T_3 via T_2 . The action-level revocation tracking exercises CI3.

6.2 Verification result

TLC 2.19 exhausts the reachable state space with the following result:

```
Model checking completed. No error has been found.
11185890 states generated, 300447 distinct states found,
    0 states left on queue.
The depth of the complete state graph search is 5.
Finished in 33s.
```

Zero violations of CI1–CI5 across all 300,447 distinct states. The fingerprint collision probability reported by TLC is 1.0×10^{-9} (based on actual fingerprints), so the exhaustive search is sound.

6.3 What the verification does and does not show

The verification establishes:

- The five invariants are consistent with the transition relation on the stated instance.

- TLC’s exhaustive search terminates with zero violations, so the theorem holds on this instance with high confidence.

The verification does *not* establish:

- The theorem for unbounded executions. The state constraint bounds $time \leq 4$ and $Len(history) \leq 3$.
- The theorem for larger instances. Scaling has been attempted but the state space grows beyond what TLC can handle; see §9.
- The theorem in a proof assistant. The proof is by hand.

The verification complements the proof: the proof is general but by hand, the verification is bounded but mechanical. Neither alone is sufficient.

7 Independence of the conditions

The three conditions are individually non-redundant. Dropping any one while keeping the other two admits a reachable state that violates a named invariant. This is the independence of the hypothesis: no condition can be removed without loss.

Method. For each condition, the artifact’s attack harness constructs a composition state that (a) satisfies the other two conditions and (b) violates the invariant tied to the dropped condition. The attack is implemented as a guard-level weakening of the TLA+ model, followed by TLC. The TLC trace is the witness.

7.1 PC non-redundancy

Dropping PC: weaken the sequence policy so it only examines the acting principal’s local chain history, not the global history. Two chains, each individually compliant, can then produce a forbidden sequence.

```
$ python3 redteam/pc/alphabet_escape.py
[PC] ATTACK SUCCEEDED - CI2 violated
[OK] PC: alphabet escape
```

The TLC trace shows a history ending in two consecutive **send** actions. CI2 is violated.

7.2 RS non-redundancy

Dropping RS: remove the guard-level revocation check. A revoked principal can then continue to act in a domain that has not yet received the revocation.

```
$ python3 redteam/rs/revocation_race.py
[RS] ATTACK SUCCEEDED - CI3 violated
[OK] RS: revocation race
```

The TLC trace shows a history entry with $h[i].time \geq r[h[i].principal]$. CI3 is violated.

This attack also serves as a *mutation test* of CI3: it demonstrates that CI3 is not vacuously satisfied. Removing the guard that enforces CI3 produces a violation, so CI3 is genuinely load-bearing.

Attack	Condition dropped	Invariant violated	Result
pc/alphabet_escape.py	PC	CI2 (Sequence Soundness)	succeeds
rs/revocation_race.py	RS	CI3 (Revocation Freshness)	succeeds
cbat/attestation_laundering.py	CBAT	CI4 (Attestation Soundness)	succeeds

Table 5: Independence of the three conditions. Each attack succeeds, so no condition can be dropped.

7.3 CBAT non-redundancy

Dropping CBAT: weaken the reachability check to allow any principal to reach any domain, regardless of attestation. A principal from a chain rooted in T_2 can then act in T_1 , which T_2 does not attest.

```
$ python3 redteam/cbat/attestation_laundering.py
[CBAT] ATTACK SUCCEEDED - CI4 violated
[OK] CBAT: attestation laundering
```

The TLC trace shows a state where the current principal’s root domain does not reach the current domain. CI4 is violated.

7.4 Summary

This is an *independence* result, not a *necessity* result. It shows the conditions are non-redundant in this construction. It does not show that no weaker condition set would suffice, and we do not claim that.

8 Related work

8.1 Delegation and access control

The speaks-for relation and its transitivity are from Lampson, Abadi, Burrows, and Wobber [1], §3.2. The same paper introduces the handoff axiom (P10) in §3.3 and notes that the general form is “too powerful” for some settings, suggesting a qualified variant—this is the mechanism our CBAT condition restates. Abadi, Burrows, Lampson, and Plotkin [2] formalize the relation into a calculus for access control in distributed systems.

SPKI/SDSI [3] provides a certificate-chain theory for authorization, with 5-tuple reduction in §6.3 and validity intervals in §5.4. RT (Li, Mitchell, and Winsborough [4]) introduces linked roles as a mechanism for bounding transitivity across domains—the closest formal analog to CBAT’s “explicitly bounded” clause.

More recently, SentinelAgent [11] provides a delegation chain calculus for AI agents with TLA+ verification. Our delegation layer restates the standard model; the composition with sequence policies and federation is the point of departure.

8.2 Sequence policies and runtime enforcement

Runtime enforcement of security policies is Schneider [5], and the safety/liveness dichotomy is Alpern and Schneider [6]. Ligatti, Bauer, and Walker [7] extend enforcement to edit automata, which can modify actions rather than only block them. Our sequence policies are safety-only and prefix-closed, so Schneider’s framework suffices.

The specific application of sequence policies to agent actions is from Bounded Agents [12] (the Agentic Principal Chain). PCAS provides a Datalog-derived policy compiler for secure agentic

systems [16]. Our PC condition is a specific constraint on the alphabet of the sequence policy; it is not a full policy language.

8.3 Federation and cross-domain trust

Federated governance without a trusted center is discussed in Wang’s testbed work [18], which addresses multi-principal agents that do not share a central controller. COA-MAS v2 [19] is a meta-framework for cross-domain governance, explicitly rejecting a single federated architecture for all contexts. Our federation graph is a specific instance of the general problem these works address; we do not claim to solve the general cross-domain governance problem.

8.4 Composition of security properties

The compositionality of safety properties is McLean [8]. Universally composable security is Canetti [9]. The composition of cryptographic protocols is addressed by Abadi and Gordon [10].

Two recent results make the negative side of our problem concrete. Spera [13] proves a non-compositionality theorem: two individually safe agents can jointly reach a forbidden goal through an emergent conjunctive dependency. AgentRFC [14] documents 20 cross-protocol violations across five composition patterns.

SynectivSec is the positive counterpart: given three specific compatibility conditions, composition preserves the invariants. Where Spera and AgentRFC establish the negative case, we characterize a positive case.

8.5 Agent-specific security frameworks

OAP (Open Agent Passport) [15] is an open specification for pre-action authorization of AI tool calls. PCAS [16] provides deterministic policy enforcement for secure agentic systems. AgentGuardian [17] learns context-aware access-control policies from execution traces. These systems operate at the per-action level; SynectivSec composes above them and does not replace them.

8.6 Summary

The individual primitives—delegation chains, sequence policies, federated attestation—have been studied extensively. What has not been stated is the *composition* of all three under a preservation theorem. That is the contribution of this paper.

9 Limitations and threats to validity

Scope of the proof

The proof of Theorem 7 is by hand. It is not mechanized in a proof assistant (Lean, Coq, Isabelle, TLAPS). A mechanized proof would strengthen the claim and is future work.

The proof assumes sequential transitions. A concurrent model with interleaved transitions would require a refinement argument not given here.

Scope of the verification

The TLA+ verification is on a bounded instance: $time \leq 4$, $Len(history) \leq 3$, 6 principals, 2 chains, 3 domains. Attempts to scale to larger instances (12 principals, or 6 principals with 4 domains and 4 capabilities) exceeded 5 minutes of runtime due to state-space explosion. Scaling

remains open and would require symmetry reduction, larger compute, or symbolic model checking (e.g., Apalache).

CI5 non-triviality

In the current model, `DomainPolicy` is a total function, so CI5 is trivially true. A model with layered or uncertain policies would make CI5 non-trivial. Extending the model to such policies is future work.

Dynamic federation

The attestation relation $\mathcal{A}$ is a compile-time constant. Domains joining or leaving the federation during a composition would require a temporal extension of the theorem.

Independence vs. necessity

The attack harness establishes independence: each condition is non-redundant in this construction. It does not establish necessity in the strong sense (that no weaker set of conditions suffices). We do not claim the stronger result.

Prior-art verification

CBAT is verified against Lampson et al. [1] §3.2 and §3.3 from the primary source. RS is verified against SPKI RFC 2693 [3] §5.4 from the primary source. PC’s lineage to Bounded Agents [12] and Schneider [5] is verified from secondary sources; a primary-source check is future work. The claim that the *joint* formulation of RS is novel has not been verified by systematic search of the 1990s–2010s trust-management literature.

No reference implementation

There is no compiled, deployable software. The TLA+ model and the attack harness are research artifacts. A reference implementation and integration with agent frameworks (MCP, A2A) is future work.

Non-goals

SynectivSec addresses action-level and sequence-level constraints. It does not address content-level attacks (prompt injection, output manipulation), availability, key compromise, or alignment. These are outside the scope of the theorem and are handled by other layers of the stack.

10 Conclusion

We have presented a composition theorem for the joint treatment of delegation chains, sequence policies, and cross-domain federation. The theorem states three compatibility conditions (PC, RS, CBAT) that are sufficient for the composition to preserve five safety invariants (CI1–CI5). We prove the theorem by structural induction over the transition relation, verify it in TLA+ on a bounded instance with zero violations across 300,447 distinct states, and demonstrate that each condition is non-redundant by a runnable counterexample.

The individual conditions are not novel. The composition is. The contribution is the joint treatment, the preservation theorem, and the independence result.

The artifact is released at <https://github.com/kakashi-kx/synergivsec> under Apache 2.0 (code) and CC-BY 4.0 (documentation). Contributions—especially counterexamples to the theorem—are welcome.

References

- [1] B. Lampson, M. Abadi, M. Burrows, and E. Wobber. Authentication in distributed systems: Theory and practice. *ACM Transactions on Computer Systems*, 10(4):265–310, 1992.
- [2] M. Abadi, M. Burrows, B. Lampson, and G. Plotkin. A calculus for access control in distributed systems. *ACM Transactions on Programming Languages and Systems*, 15(4):706–734, 1993.
- [3] C. Ellison, B. Frantz, B. Lampson, R. Rivest, B. Thomas, and T. Ylonen. SPKI certificate theory. RFC 2693, Internet Engineering Task Force, September 1999.
- [4] N. Li, J. C. Mitchell, and W. H. Winsborough. Design of a role-based trust-management framework. In *Proc. IEEE Symposium on Security and Privacy*, 2002.
- [5] F. B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000.
- [6] B. Alpern and F. B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985.
- [7] J. Ligatti, L. Bauer, and D. Walker. Edit automata: Enforcement mechanisms for run-time security policies. *International Journal of Information Security*, 4(1–2):2–16, 2005.
- [8] J. McLean. A general theory of composition for trace sets closed under selective interleaving functions. In *Proc. IEEE Symposium on Security and Privacy*, 1994.
- [9] R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proc. IEEE Symposium on Foundations of Computer Science*, 2001.
- [10] M. Abadi and A. D. Gordon. A calculus for cryptographic protocols: The spi calculus. In *Proc. ACM Conference on Computer and Communications Security*, 1997.
- [11] Anonymous. SentinelAgent: A delegation chain calculus for AI agents. arXiv:2604.02767, 2026.
- [12] Anonymous. Bounded agents: The agentic principal chain. arXiv:2608.15888, 2026.
- [13] A. Spera. Non-compositionality of safety for AI agents. arXiv:2603.15973, 2026.
- [14] Anonymous. AgentRFC: Composition safety for agent protocols. arXiv:2603.23801, 2026.
- [15] U. Uchibeke. Open Agent Passport: Pre-action authorization for AI tool calls. arXiv:2603.20953, 2026.
- [16] N. Palumbo, S. Choudhary, J. Choi, P. Chalasani, and S. Jha. Policy compiler for secure agentic systems. arXiv:2602.16708, 2026.
- [17] N. Abaev et al. AgentGuardian: Learning access control policies to govern AI agent behavior. arXiv:2601.10440, 2026.
- [18] K. Wang. A federated governance testbed for multi-principal agents. 2026.
- [19] Anonymous. COA-MAS v2: A meta-framework for cross-domain agent governance. 2026.

A Full proof of the preservation theorem

We give the full structural induction. The notation follows §3.

A.1 Base case

Let $s_0 = \langle d_0, \varepsilon, T_0, r_0, 0 \rangle$.

CI1. $|h| = 0$, so the universal quantifier is vacuous.

CI2. φ is prefix-closed, and ε is a prefix of every history, so $\varepsilon \models \varphi$.

CI3. $|h| = 0$, vacuous.

CI4. $T_0 = \text{RootDomain}(p_0)$ by definition of s_0 , and $T_0 \in \text{Reach}(F, T_0)$ by the reflexive base case.

CI5. DomainPolicy is total by definition.

A.2 Inductive step: T-Action

$s \rightarrow s'$ where $s' = \langle d, h \smallfrown \langle \langle p, a, t \rangle \rangle, T, r, t + 1 \rangle$, subject to G1–G5.

CI1. For $i \leq |h|$, $h'[i] = h[i]$, so CI1(s) applies. For $i = |h| + 1$, $h'[i].\text{action} = a \in \kappa(p)$ by G2. So CI1 holds.

CI2. $h' = h \smallfrown \langle a \rangle \models \varphi$ by G3.

CI3. For $i \leq |h|$, $h'[i].\text{time} = h[i].\text{time} < r[h[i].\text{principal}]$ by CI3(s) and $r' = r$. For $i = |h| + 1$, $h'[i].\text{time} = t < r[p]$ by G2, and $h'[i].\text{principal} = p$, so the inequality holds.

CI4. $T \in \text{Reach}(F, \text{RootDomain}(p))$ by G1, and p is now the current principal.

CI5. T and the historical actions are unchanged; $a \in \Sigma$ by PC; DomainPolicy is total.

A.3 Inductive step: T-Revoke

$s \rightarrow s'$ where $s' = \langle d, h, T, r[p \mapsto t], t + 1 \rangle$, subject to $r[p] = \infty$.

CI1, CI2, CI4, CI5. History and current domain unchanged; immediate.

CI3. Let $i \leq |h|$. If $h[i].\text{principal} \neq p$, then $r'[h[i].\text{principal}] = r[h[i].\text{principal}]$ and CI3(s) applies. If $h[i].\text{principal} = p$, we have $h[i].\text{time} < t$ because actions are recorded at strictly earlier times than t , so $h[i].\text{time} < t = r'[p]$.

A.4 Inductive step: T-Cross

$s \rightarrow s'$ where $s' = \langle p, h, T', r, t + 1 \rangle$, subject to $T' \in \text{Reach}(F, \text{RootDomain}(p))$.

CI1, CI2, CI3. History unchanged.

CI4. $T' \in \text{Reach}(F, \text{RootDomain}(p))$ by the guard, and p is now the current principal.

CI5. Unchanged.

A.5 Conclusion

By structural induction, every reachable state $s \in \text{Reach}(D \otimes S \otimes F)$ satisfies CI1–CI5. $\square$

B TLA+ module

The complete TLA+ module is in the artifact at `tla/Composition.tla`. We reproduce the invariant definitions and the transition relation here for reference.

```
DoAction(p, a, dom) ==
  /\ ChainRootCanReachDomain(p, dom)
  /\ ActionPermitted(p, a, dom)
  /\ Len(history) < MaxHistory
  /\ SequencePolicyOK(Append(history,
    [principal |-> p, action |-> a, at |-> time]))
  /\ currentPrincipal' = p
  /\ history' = Append(history,
    [principal |-> p, action |-> a, at |-> time])
  /\ currentDomain' = dom
  /\ revokedAt' = revokedAt
  /\ time' = time + 1

Revoke(p) ==
  /\ revokedAt[p] = NeverRevoked
  /\ revokedAt' = [revokedAt EXCEPT ![p] = time]
  /\ currentPrincipal' = currentPrincipal
  /\ history' = history
  /\ currentDomain' = currentDomain
  /\ time' = time + 1

CrossDomain(p, dom) ==
  /\ ChainRootCanReachDomain(p, dom)
  /\ currentPrincipal' = p
  /\ history' = history
  /\ currentDomain' = dom
  /\ revokedAt' = revokedAt
  /\ time' = time + 1
```

The full module includes the capability lattice, the attestation relation, the sequence policy, and the five invariants.